

Centro Universitário Católica SC

Engenharia de Software — 5ª Fase

Qualidade de Software

BarberManager

Análise de Qualidade de Código
com SonarQube Community Build

Miguel Angel Balladares Huertas

13 de junho de 2026

Repositório do Projeto

<https://github.com/miguettahuertas/BarberManager>

Sumário

1	Identificação do Projeto	1
1.1	Nome do Projeto	1
1.2	Objetivo do Sistema	1
1.3	Tecnologias Utilizadas	1
2	Configuração da Análise	1
2.1	Versão do SonarQube	1
2.2	Linguagem Analisada	2
2.3	Forma de Execução	2
2.4	Data da Análise	2
3	Resultados Obtidos	3
3.1	Visão Geral (Overview)	3
3.2	Métricas Detalhadas	4
3.2.1	Nota sobre Duplicações	4
4	Análise dos Problemas Encontrados	5
4.1	Code Smell 1 — Contraste Insuficiente de Texto	5
4.2	Code Smell 2 — Uso de <a> como Botão	5
4.3	Code Smell 3 — Elemento sem Atributo <code>aria-label</code>	6
4.4	Problema de Segurança — Resource Integrity (Web:S5725)	7
4.5	Problema de Duplicação — 100% de Código Duplicado	8
5	Plano de Correção	8
5.1	Correção 1 — Adicionar SRI aos Recursos Externos (Alta)	8
5.2	Correção 2 — Resolver Duplicação Arquitetural (Alta)	9
5.3	Correção 3 — Melhorar Contraste de Cores (Média)	9
5.4	Correção 4 — Substituir <a> por <button> (Média)	10
5.5	Correção 5 — Adicionar Atributos ARIA (Baixa)	10
6	Conclusão	10
6.1	Utilidade do SonarQube no Processo de Desenvolvimento	10
6.2	Métricas Mais Relevantes	11
6.3	Principais Problemas Encontrados	11

1 Identificação do Projeto

1.1 Nome do Projeto

BarberManager — Sistema de Gestão para Barbearias

1.2 Objetivo do Sistema

O BarberManager é um sistema completo de gestão para barbearias, desenvolvido como projeto académico da disciplina de Programação Web (5ª Fase — Engenharia de Software). O sistema oferece dois módulos principais:

- **App Mobile/Desktop (Clientes):** cadastro, autenticação, catálogo dinâmico de serviços, agendamento completo com escolha de barbeiro e horário, visualização de agendamentos históricos e avaliação pós-serviço com sistema de estrelas.
- **Dashboard Web/Desktop (Administradores):** KPIs financeiros em tempo real, linha do tempo da agenda do dia com auto-atualização de status, gestão CRUD completa de serviços, clientes, relatórios financeiros e configurações da barbearia.

O sistema está publicado online com acesso público via ngrok, disponível no endereço <https://humid-boots-posted.ngrok-free.dev>.

1.3 Tecnologias Utilizadas

Camada	Tecnologia
Linguagem principal	Delphi 12 Athens (Object Pascal)
UI Framework	FireMonkey (FMX) — cross-platform nativo
REST API	Horse Framework — servidor HTTP porta 9000
Frontend Web	HTML5 / CSS3 / JavaScript puro (SPA single-file)
Base de Dados	Firebird 3.0 (SGBD relacional)
Acesso a Dados	FireDAC — componentes criados 100% por código
Deploy	ngrok Tunnel — URL pública permanente
Controlo de Versão	Git + GitHub
Qualidade de Código	SonarQube Community Build v26.6.0.123539

Tabela 1: Stack tecnológica do BarberManager

2 Configuração da Análise

2.1 Versão do SonarQube

SonarQube Community Build v26.6.0.123539 — versão gratuita instalada localmente em C:\sonarqube\sonarqube-26.6.0.123539.

2.2 Linguagem Analisada

A análise incidiu sobre os ficheiros do **frontend web** do projeto:

- `src/Web/index.html` — SPA completa (HTML5 + CSS3 + JavaScript) com 2.132 linhas
- `index.html` — cópia servida pelo servidor Horse em GET /

O código Delphi (ficheiros `.pas`) não possui suporte nativo no SonarQube Community, pelo que a análise focou-se no frontend web — componente igualmente relevante do sistema.

2.3 Forma de Execução

A análise foi executada via **SonarScanner CLI v8.0.1.6346** a partir da linha de comandos, com o ficheiro `sonar-project.properties` configurado na raiz do repositório:

Listing 1: `sonar-project.properties`

```
1 sonar.projectKey=barbermanager
2 sonar.projectName=BarberManager
3 sonar.projectVersion=1.0
4 sonar.sources=src
5 sonar.inclusions=**/*.js,**/*.html,**/*.pas,**/*.dpr
6 sonar.host.url=http://localhost:9000
7 sonar.sourceEncoding=UTF-8
8 sonar.exclusions=**/node_modules/**,**/.git/**,**/Win32/**,**/
   Android/**
```

Comando de execução:

Listing 2: Comando de execução do SonarScanner

```
1 "C:\sonar-scanner\...\bin\sonar-scanner.bat"
2 -D"sonar.projectKey=BarberManager"
3 -D"sonar.sources=."
4 -D"sonar.host.url=http://localhost:9000"
5 -D"sonar.token=sqp_c4ef070f992044231bee8e52e77839eeb689c749"
```

2.4 Data da Análise

13 de junho de 2026 — 15:53

Tempo total de análise: 40,995 segundos

Revisão Git: 70655a5837c646e1248df9e305fd9108ae21bec2

3 Resultados Obtidos

3.1 Visão Geral (Overview)

Quality Gate — Resultado Final

✓ PASSED

O projeto passou no Quality Gate com sucesso. Todos os critérios mínimos de qualidade foram satisfeitos na análise da versão 1.0.

Dimensão	Rating	Issues	Interpretação
Security	B	2	Bom — 1 vulnerabilidade de baixa severidade
Reliability	C	72	Aceitável — problemas de acessibilidade e semântica HTML
Maintainability	A	60	Excelente — dívida técnica baixa (6h 24min)
Coverage	—	—	Sem cobertura de testes (ausência de testes automatizados)
Duplications	100%	—	Crítico — todo o código é considerado duplicado

Tabela 2: Resumo das dimensões de qualidade

3.2 Métricas Detalhadas

Métrica	Valor	Observação
Linhas de Código (LOC)	4.264	HTML: 4.3k (2 ficheiros analisados)
Linhas totais	4.800	Incluindo linhas em branco e comentários
Ficheiros analisados	2	<code>src/Web/index.html</code> + <code>index.html</code>
Linhas de comentário	46	1,1% do total
Cobertura de Testes	—	Ausente (0% — sem testes automatizados)
Complexidade Ciclomática	N/D	Não reportada para HTML puro
Complexidade Cognitiva	N/D	Não reportada para HTML puro
Duplicação de Código	100%	4.798 linhas duplicadas / 2 blocos / 2 ficheiros
Bugs	72	Rating C — problemas de acessibilidade/semântica
Vulnerabilidades	2	Rating B — integridade de recursos externos
Security Hotspots	0	Nenhum hotspot identificado
Code Smells	60	Rating A — dívida técnica de 6h 24min
Dívida Técnica (Technical Debt)	6h 24min	Debt Ratio: 0.3%
Índice de Manutenibilidade	A	Excelente — esforço zero para atingir rating A
Esforço de Fiabilidade	5h 42min	Tempo estimado para corrigir os 72 bugs

Tabela 3: Métricas completas da análise SonarQube

3.2.1 Nota sobre Duplicações

A duplicação de 100% deve-se ao facto de o projeto ter **dois ficheiros HTML idênticos**: `src/Web/index.html` (fonte) e `index.html` (cópia na raiz, servida pelo servidor Horse). O SonarQube detectou os 4.798 blocos duplicados entre estes dois ficheiros. Esta duplicação é **intencional e arquitetural** — a cópia na raiz é gerada automaticamente por um post-build MSBuild — e não representa duplicação real de lógica.

4 Análise dos Problemas Encontrados

4.1 Code Smell 1 — Contraste Insuficiente de Texto

Code Smell: Contraste de Cores (css:S7924)

Descrição: O texto não cumpre os requisitos mínimos de contraste em relação ao fundo, violando as diretrizes WCAG (Web Content Accessibility Guidelines).

Ficheiro: src/Web/index.html — Linha 83 e múltiplas ocorrências

Severidade: Maintainability — Medium

Esforço: 5 minutos por ocorrência

Trecho de código afetado:

Listing 3: Linha 83 — index.html

```
1 .auth-success {  
2     background: rgba(34,197,94,.12);  
3     border: 1px solid rgba(34,197,94,.3);  
4     border-radius: 8px; color: #86EFAC;  
5     font-size: 13px; padding: 10px 14px; margin-bottom: 16px;  
6 }
```

Por que o SonarQube considera um problema? A cor #86EFAC (verde claro) sobre o fundo rgba(34,197,94,.12) (verde translúcido sobre fundo escuro #0B1220) não atinge a relação de contraste mínima de **4.5:1** exigida pelas WCAG 2.1 para texto normal. Utilizadores com baixa visão ou daltonismo podem não conseguir ler o conteúdo.

Impacto: Acessibilidade comprometida; em contextos profissionais ou governamentais, pode representar violação de normas de acessibilidade digital.

4.2 Code Smell 2 — Uso de <a> como Botão

Code Smell: Âncora usada como botão (Web:S6844)

Descrição: Elementos <a> (âncoras/links) estão a ser utilizados como botões, sem href válido e sem função de navegação.

Ficheiro: src/Web/index.html — Linha 684 e múltiplas ocorrências

Severidade: Reliability — Low / Maintainability — Medium

Esforço: 5 minutos por ocorrência

Trecho de código afetado:

Listing 4: Linha 684 — index.html

```
1 <!-- CADASTRO -->
2 <div id="screen-cadastro" class="auth-screen" style="display:none">
3   <div class="auth-card">
4     <div class="auth-logo">&lt;&lt;/div>
5     <h1>Criar Conta</h1>
6     <p class="auth-sub">Cadastre-se gratuitamente</p>
```

Por que o SonarQube considera um problema? Tags `<a>` sem `href` são semanticamente âncoras sem destino. Leitores de ecrã (screen readers) tratam-nas de forma diferente de botões, prejudicando a navegação por teclado e a acessibilidade. A regra `Web:S6844` exige que elementos com comportamento de botão utilizem a tag `<button>` ou incluam `role="button"`.

Impacto: Acessibilidade e manutenibilidade reduzidas; comportamento inconsistente entre browsers; dificulta navegação por teclado (Tab/Enter).

4.3 Code Smell 3 — Elemento sem Atributo `aria-label`

Code Smell: Elemento sem `aria-label` (Web:S5255)

Descrição: Elemento interativo sem atributo `aria-label` ou `aria-labelledby`, tornando-o inacessível para tecnologias assistivas.

Ficheiro: `src/Web/index.html` — Linha 726

Severidade: Maintainability — Medium

Esforço: 5 minutos

Trecho de código afetado:

Listing 5: Linha 726-735 — index.html

```
1 <div class="sidebar-logo">
2   <div class="logo-icon">&lt;&lt;/div>
3   <div>                                <!-- Linha 729: sem aria-label -->
4     <div class="logo-title">BarberManager</div>
5     <div class="logo-sub">Painel Admin</div>
6   </div>
7 </div>
8 <div class="nav-label">Principal</div>
```

Por que o SonarQube considera um problema? Elementos `<div>` com papel interativo implícito devem ter labels ARIA para que leitores de ecrã possam descrevê-los. Sem `aria-label`, utilizadores com deficiência visual não conseguem identificar o propósito do elemento.

Impacto: Acessibilidade comprometida; não conformidade com WCAG 2.1 Critério 4.1.2 (Nome, Função, Valor).

4.4 Problema de Segurança — Resource Integrity (Web:S5725)

Vulnerabilidade de Segurança: Integridade de Recursos Externos (Web:S5725)

Descrição: Recursos externos (scripts, estilos) carregados sem o atributo `integrity` (Subresource Integrity — SRI).

Ficheiro: `src/Web/index.html` — Linha 9

Severidade: Security — Low

Esforço: 5 minutos

Trecho de código afetado:

Listing 6: Linha 9 — `index.html` (estilo atual)

```
1 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs
  /
2 font-awesome/6.4.0/css/all.min.css">
3 <!-- Sem atributo integrity= -->
```

Versão correta com SRI:

Listing 7: Versão corrigida com integrity check

```
1 <link rel="stylesheet"
2 href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/
3 6.4.0/css/all.min.css"
4 integrity="sha512-iecdLmaskl7CVkqkXNQ/ZH/
  XLlvWZOJyj7Yy7tcenmpD1ypASozpmT/E0iPtmFIB46ZmdtAc9eNBvH0H/
  ZpiBw=="
5 crossorigin="anonymous">
```

Por que o SonarQube considera um problema? Sem o atributo `integrity`, um atacante que comprometa o CDN (Content Delivery Network) pode substituir o ficheiro por código malicioso. O browser executa o ficheiro sem verificar se foi adulterado. Com SRI, o hash garante que apenas o ficheiro original é aceite.

Impacto: Vulnerabilidade a ataques de supply-chain; um CDN comprometido pode injetar código malicioso nos navegadores de todos os utilizadores do sistema.

4.5 Problema de Duplicação — 100% de Código Duplicado

Duplicação: 100% de Densidade (4.798 linhas)

Descrição: O SonarQube detectou 4.798 linhas duplicadas distribuídas em 2 blocos e 2 ficheiros, resultando numa densidade de duplicação de 100%.

Ficheiros envolvidos:

- `src/Web/index.html` — 2.132 linhas
- `index.html` (raiz) — cópia idêntica servida pelo servidor

Causa: Arquitetura de deploy — o ficheiro fonte é copiado para a raiz automaticamente pelo post-build MSBuild para ser servido pelo Horse em `GET /`.

Por que o SonarQube considera um problema? O SonarQube compara o conteúdo dos ficheiros sem conhecer a intenção arquitetural. Ao encontrar dois ficheiros com conteúdo idêntico, classifica-os como duplicação, independentemente de serem intencionais. Em projetos reais sem esta justificação, 100% de duplicação indicaria sérios problemas de organização de código.

Impacto (se fosse duplicação real): Manutenção dobrada — qualquer correção teria de ser replicada em ambos os ficheiros; risco de inconsistências entre versões.

5 Plano de Correção

#	Problema	Prioridade	Melhoria Esperada
1	Integridade de recursos (SRI)	Alta	Elimina vulnerabilidade de supply-chain
2	Duplicação de ficheiros	Alta	Reduz duplicação de 100% para 0%
3	Contraste insuficiente	Média	Melhora acessibilidade WCAG 2.1
4	<code><a></code> usado como botão	Média	Melhora semântica e navegação por teclado
5	Elementos sem <code>aria-label</code>	Baixa	Melhora suporte a leitores de ecrã

Tabela 4: Plano de correção por prioridade

5.1 Correção 1 — Adicionar SRI aos Recursos Externos (Alta)

Como corrigir: Gerar o hash SHA-512 de cada recurso externo e adicioná-lo ao atributo `integrity`:

Listing 8: Recursos externos com SRI

```
1 <!-- Font Awesome com integrity check -->
2 <link rel="stylesheet"
3   href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/
4     css/all.min.css"
5   integrity="sha512-iecdLmaskl7CVkqkXNQ/..."
6   crossorigin="anonymous"
   referrerpolicy="no-referrer">
```

Ferramenta: Usar srihash.org para gerar o hash de cada URL.

Melhoria: Elimina as 2 vulnerabilidades de segurança; rating Security sobe de B para A.

5.2 Correção 2 — Resolver Duplicação Arquitetural (Alta)

Como corrigir: Excluir o ficheiro `index.html` da raiz da análise SonarQube, uma vez que é uma cópia intencional:

Listing 9: sonar-project.properties atualizado

```
1 # Excluir a copia de deploy da analise
2 sonar.exclusions=**/node_modules/**,**/.git/**,**/Win32/**,
3   **/Android/**,**/*.dcu,**/*.dproj,**/*.res,
4   **/*.identcache,index.html
```

Melhoria: Reduz duplicação de 100% para 0%; análise mais precisa do código real.

5.3 Correção 3 — Melhorar Contraste de Cores (Média)

Como corrigir: Aumentar o contraste das cores de texto para atingir ratio mínimo de 4.5:1:

Listing 10: CSS corrigido — contraste adequado

```
1 /* Antes */
2 .auth-success { color: #86EFAC; }
3
4 /* Depois -- contraste 4.5:1 garantido */
5 .auth-success { color: #15803D; } /* Verde escuro WCAG AA */
```

Ferramenta: Usar [WebAIM Contrast Checker](https://webaim.org/contrast-checker).

Melhoria: Reduz Code Smells de contraste (estimado: 20 ocorrências); melhora acessibilidade.

5.4 Correção 4 — Substituir <a> por <button> (Média)

Como corrigir: Substituir todas as âncoras usadas como botões pela tag semântica correta:

Listing 11: HTML semântico correto

```
1 <!-- Antes (incorreto) -->
2 <a onclick="mostrarTela('login')">Voltar ao login</a>
3
4 <!-- Depois (correto) -->
5 <button type="button" onclick="mostrarTela('login')">
6     class="btn-link">Voltar ao login</button>
```

Melhoria: Melhora semântica HTML; navegação por teclado funcional; reduz bugs de Reliability.

5.5 Correção 5 — Adicionar Atributos ARIA (Baixa)

Como corrigir: Adicionar `aria-label` a todos os elementos interativos sem texto descritivo:

Listing 12: ARIA labels adicionados

```
1 <!-- Antes -->
2 <div class="sidebar-logo">
3
4 <!-- Depois -->
5 <div class="sidebar-logo" role="banner"
6     aria-label="Logo BarberManager -- Painel Admin">
```

Melhoria: Conformidade WCAG 2.1 Critério 4.1.2; suporte completo a leitores de ecrã.

6 Conclusão

6.1 Utilidade do SonarQube no Processo de Desenvolvimento

A aplicação do SonarQube ao projeto BarberManager demonstrou o valor da análise estática de código como componente essencial do ciclo de desenvolvimento. A ferramenta identificou **94 issues** numa única execução, abrangendo problemas de segurança, acessibilidade e manutenibilidade que seriam difíceis de detectar manualmente numa SPA com mais de 4.000 linhas de código.

A integração via `sonar-project.properties` no repositório Git permite que a análise seja reproduzível em qualquer ambiente, facilitando a adoção de práticas de *Continuous*

Quality. Em equipas maiores, esta configuração poderia ser integrada num pipeline CI/CD (GitHub Actions, Jenkins) para análise automática a cada *pull request*.

6.2 Métricas Mais Relevantes

As métricas consideradas mais relevantes para o contexto deste projeto foram:

1. **Security (Rating B, 2 issues):** A vulnerabilidade de integridade de recursos externos é a mais crítica em termos de impacto real, pois representa um vetor de ataque concreto (supply-chain attack via CDN comprometido).
2. **Duplications (100%):** Embora neste caso seja intencional e arquitetural, a métrica de duplicação é fundamental para detetar código morto e violações do princípio DRY em projetos reais.
3. **Maintainability (Rating A, 6h 24min de dívida):** O rating A com dívida técnica reduzida confirma que o código frontend está bem estruturado e é fácil de manter, apesar do elevado número de Code Smells de acessibilidade.
4. **Coverage (ausente):** A ausência total de cobertura de testes é o ponto mais crítico para a evolução futura do sistema. Sem testes automatizados, qualquer refatoração carrega risco elevado de regressão.

6.3 Principais Problemas Encontrados

Os principais problemas identificados na análise foram:

- **Acessibilidade (72 Reliability issues):** A grande maioria dos problemas relaciona-se com violações das diretrizes WCAG — contraste insuficiente, uso semântico incorreto de `<a>` como botão, e ausência de atributos ARIA. Estes problemas afetam utilizadores com deficiências visuais e navegação por teclado.
- **Segurança (2 Security issues):** Recursos externos carregados sem verificação de integridade (SRI), expondo o sistema a ataques de supply-chain.
- **Duplicação arquitetural (100%):** Consequência do padrão de deploy adotado (cópia do HTML para a raiz), facilmente resolvível com uma exclusão na configuração do SonarQube.
- **Ausência de testes:** O zero em Coverage representa o maior gap de qualidade do projeto para produção real.

Em síntese, o BarberManager apresenta uma base de código de qualidade razoável para um projeto acadêmico — Quality Gate **Passed**, Maintainability **A**, sem Security Hotspots. Os problemas identificados são predominantemente de acessibilidade e podem ser corrigidos de forma sistemática com as propostas apresentadas neste relatório.

Miguel Angel Balladares Huertas
Engenharia de Software — 5^a Fase
Centro Universitário Católica SC
13 de junho de 2026